Avaliação Somativa II

Aluno: César Willian Pacheco

Suponha que você foi contratado por uma empresa que precisa de um modelo preditivo para um problema de classificação de tipos de vidro, denominado **Glass**.

A empresa preparou uma base de dados com 214 instâncias, cada uma com 11 atributos numéricos. O primeiro atributo deve ser ignorado, pois é o ID da instância; em seguida, seguem 9 atributos de entrada numéricos e a classe.

Cada atributo de entrada diz respeito à composição do vidro (índice de refração e quantidades de sódio, magnésio, alumínio, sílica, potássio, cálcio, bário e ferro).

Há 7 classes de vidros, sendo: 1,2,3,4,5,6,7. Contudo, não há exemplos da classe 4 na base. Isto não impede a construção de classificadores para o problema.

CUIDADO !!: Ao alterar a parte do script (Compara_classifiers) que mostra a matriz de confusão, utilize "123567". Observe que a classe 4 não aparecerá na matriz de confusão.

Apenas para seu conhecimento, segue o nome de cada classe:

1 building_windows_float_processed
2 building_windows_non_float_processed
3 vehicle_windows_float_processed
4 vehicle_windows_non_float_processed (sem exemplos na base)
5 containers
6 tableware
7 headlamps

Leitura da base:

```
# Ler direto da URL
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/glass/glass.data"
raw_data = urllib.request.urlopen(url)

# Carrega arquivo como uma matriz
dataset = np.loadtxt(raw_data, delimiter=",")

# Separa atributos de entrada em X e as classes em y
# Já ignora o ID da instância
X = dataset[:,1:10]
y = dataset[:,10]
```

A) DESAFIO: Encontrar a melhor solução para este problema através da avaliação de soluções monolíticas (uso de um único classificador) e ensembles (uso de comitês de classificadores). Anote na tabela abaixo os melhores resultados encontrados para cada abordagem.

Tabela de Resultados (Holdout 60/40)

Classificadores Monolíticos	**Taxa de Acerto (%)**
Árvores de Decisão	61.63
KNN	**82.56**
KNN weighted (com pesos)	75.58
Naive Bayes	51.16
MLP	69.77
SVM	63.95
Ensembles	**Taxa de Acerto (%)**
Ensemble heterogêneo usando voto majoritário *Membros: Árvore de Decisão + KNN weighted + SVM + MLP (cada um com os hiperparâmetros da parte monolítica); voto majoritário (hard)*	72.09
Ensemble heterogêneo usando voto Soma/Max) *Membros: Árvore de Decisão + KNN weighted + SVM + MLP (cada um com os hiperparâmetros da parte monolítica); soma das probabilidades (soft)*	79.07
Ensemble homogêneo com Bagging	81.40
Ensemble homogêneo com Boosting (Adaboost)	79.07
Ensemble homogêneo com Random Forest	80.23

*** DESTAQUE EM NEGRITO O MELHOR RESULTADO

Protocolo: holdout 60/40 estratificado (train_test_split, test_size=0.40, random_state=42, stratify=y) → 128 instâncias de treinamento e 86 de teste, idênticas para todos os classificadores. Os hiperparâmetros de cada técnica foram escolhidos por GridSearchCV (validação cruzada estratificada com 5 folds) usando somente a porção de treinamento; a taxa de acerto reportada é a medida uma única vez na porção de teste. KNN, SVM e MLP usam StandardScaler em Pipeline (ajustado apenas no treino). Os hiperparâmetros de todos os classificadores estão no Apêndice A.

B) CONSIDERANDO O SEU MELHOR RESULTADO, APRESENTE:

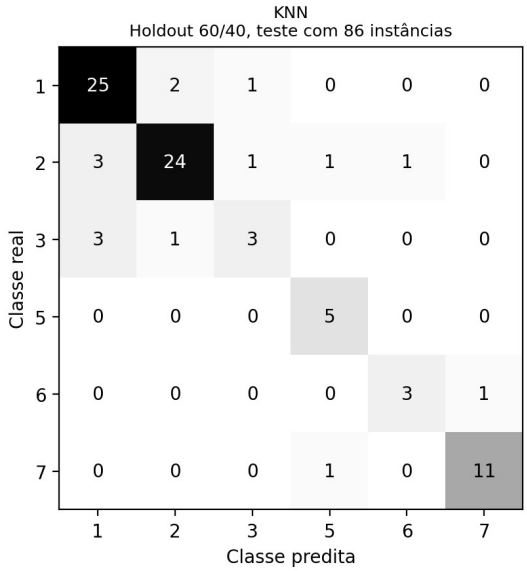
B.1) Os parâmetros utilizados para que o professor possa reproduzir seus resultados.
Melhor resultado: KNN com taxa de acerto de 82.56% na porção de teste (71 acertos em 86 instâncias).
Reprodução (scikit-learn 1.8, Python 3):

```
RANDOM_STATE = 42
X = dataset[:, 1:10];  y = dataset[:, 10]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.40,
                                   random_state=42, stratify=y)
clf = Pipeline([('scaler', StandardScaler()),
                ('clf', KNeighborsClassifier(n_neighbors=1, weights='uniform',
p=1))])
clf.fit(X_train, y_train);  acc = clf.score(X_test, y_test)   # 0.8256
```

Parâmetros: n_neighbors = 1 (1-NN), weights = 'uniform', p = 1 (distância de Manhattan / Minkowski com r = 1), metric = 'minkowski', algorithm = 'auto'; atributos padronizados com StandardScaler (média 0 e desvio

1 calculados no treino). A grade testada foi n_neighbors ∈ {1, 3, 5, 7, 9} × p ∈ {1, 2}, selecionada por acurácia média em validação cruzada de 5 folds dentro do treino.
Vale notar que os ensembles homogêneos ficaram muito próximos (Bagging 81,40%, Random Forest 80,23%, AdaBoost 79,07%) e são mais estáveis que um 1-NN; com 86 instâncias de teste, uma diferença de 1 ponto percentual corresponde a um único exemplo. O 1-NN funciona bem aqui porque a base é pequena, os atributos são poucos e numéricos e vidros de composição quase idêntica pertencem à mesma classe (relação local entre atributos e classe).

B.2) A matriz de confusão.



Matriz de confusão do KNN (1-NN) na porção de teste (86 instâncias). Linhas = classe real, colunas = classe predita; rótulos 1, 2, 3, 5, 6 e 7 (a classe 4 não possui exemplos).

real \ predita	**1**	**2**	**3**	**5**	**6**	**7**
1	**25**	2	1	0	0	0
2	3	**24**	1	1	1	0
3	3	1	**3**	0	0	0
5	0	0	0	**5**	0	0
6	0	0	0	0	**3**	1
7	0	0	0	1	0	**11**

B.3) Qual é a taxa de acerto da classe com mais erros?

Resposta: A classe com mais erros é a classe 2 (building_windows_non_float_processed), com 6 erros em 30 instâncias de teste: taxa de acerto de 24/30 = 80,00%. Seus erros se espalham por quase todas as outras classes (3 exemplos para a classe 1, 1 para a 3, 1 para a 5 e 1 para a 6).
Se considerarmos a MENOR taxa de acerto (proporção de erros), a pior classe é a 3 (vehicle_windows_float_processed): 3/7 = 42,86%, com 4 erros, sendo 3 confundidos com a classe 1 e 1 com a classe 2. Isso é esperado: as classes 1, 2 e 3 são todas vidros de janela com composições muito parecidas, e a classe 3 é minoritária (17 exemplos na base inteira, 7 no teste). Taxa de acerto por classe: 1 = 25/28 (89,29%); 2 = 24/30 (80,00%); 3 = 3/7 (42,86%); 5 = 5/5 (100%); 6 = 3/4 (75,00%); 7 = 11/12 (91,67%). Taxa de acerto geral = 71/86 = 82,56%.

Apêndice A: Hiperparâmetros selecionados para cada classificador

Classificador	**Taxa de acerto (%)**	**Hiperparâmetros (GridSearchCV, 5 folds no treino)**
Arvore de Decisao	61.63	criterion=gini, max_depth=None, min_samples_leaf=1, min_samples_split=2
KNN	**82.56**	n_neighbors=1, p=1

KNN weighted	75.58	n_neighbors=5, p=1
Naive Bayes	51.16	var_smoothing=0.001
MLP	69.77	activation=tanh, alpha=0.01, hidden_layer_sizes=(16,)
SVM	63.95	C=1000, gamma=0.01, kernel=rbf
Ensemble heterogeneo com voto majoritario (DT+KNNw+SVM+MLP)	72.09	membros = melhores monoliticos; voting=hard
Ensemble heterogeneo com soma de probabilidades soft (DT+KNNw+SVM+MLP)	79.07	membros = melhores monoliticos; voting=soft
Bagging (Arvores)	81.40	max_samples=1.0, n_estimators=100
AdaBoost (Arvores)	79.07	learning_rate=0.5, n_estimators=50
Random Forest	80.23	max_depth=None, max_features=sqrt, n_estimators=500

Apêndice B: Script utilizado

Os experimentos foram executados localmente em Python 3 com scikit-learn (não foi utilizado o Google Colab); por isso o script acompanha esta entrega como arquivo separado: atividade3_holdout.py (junto com a saída completa da execução em saida_atividade3.txt e uma cópia local da base, glass.data, usada automaticamente caso a URL do UCI não esteja acessível). Para reproduzir os resultados:

```
python atividade3_holdout.py
```